Relatório

Alunos:

- João Vitor Garcia Aguiar Mintz; 10440421
- Mateus Kage Moya; 10332608

Introdução:

Neste relatório, apresentamos o desenvolvimento do Projeto, no qual tivemos como objetivo construir um analisador de logs de sensores IoT utilizando a linguagem C em conjunto com a biblioteca Pthreads. A proposta do trabalho foi baseada em um cenário semelhante ao de um data center, em que diversos sensores geram leituras de forma contínua, exigindo um processamento eficiente para a obtenção de informações estatísticas importantes.

O projeto foi dividido em etapas, de modo que pudéssemos comparar diferentes abordagens de implementação. Inicialmente, desenvolvemos uma versão sequencial, utilizada como base para análise. Em seguida, implementamos uma versão paralela com uso de mutex, com a finalidade de proteger as seções críticas durante o acesso concorrente aos dados compartilhados. Por fim, elaboramos uma versão paralela otimizada, na qual buscamos reduzir a contenção causada pelos mutex por meio do uso de acumuladores locais em cada thread.

Com isso, foi possível analisar não apenas os ganhos de desempenho proporcionados pela paralelização, mas também as dificuldades e limitações relacionadas à sincronização entre threads. As métricas consideradas no projeto foram a média das leituras por sensor, a identificação do sensor mais instável com base no maior desvio padrão, a contagem total de alertas com status **ALERTA** ou **CRÍTICO**, e o cálculo do consumo total de energia.

Etapa 1: Geração dos dados:

Parâmetros usados:

Número de sensores: 100

Número de dias: 2

Total de linhas: 17.280.000

Probabilidade de anomalias: 0,01 (1%)

Características do arquivo gerado:

Tamanho em MB: 934,76 MB

Distribuição de sensores por tipo: temperatura: 41, umidade: 24, energia: 20, corrente: 14, pressão: 1

Linha de comando utilizada: **python3 scriptProfessor/generate_sensor_log.py --sensores 100 --dias 2 --output sensores.log**

Print das 10 primeiras linhas para verificar formato:

```
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ date
Tue Mar 31 12:06:27 -03 2026
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ head -10 sensores.log
sensor_001 2026-03-01 00:00:00 corrente 2.9 status OK
sensor_006 2026-03-01 00:00:00 corrente 2.0 status OK
sensor_012 2026-03-01 00:00:00 corrente 2.3 status OK
sensor_015 2026-03-01 00:00:00 corrente 2.5 status OK
sensor_030 2026-03-01 00:00:00 corrente 2.8 status OK
sensor_033 2026-03-01 00:00:00 corrente 3.0 status OK
sensor_040 2026-03-01 00:00:00 corrente 3.9 status OK
sensor_042 2026-03-01 00:00:00 corrente 3.5 status OK
sensor_045 2026-03-01 00:00:00 corrente 3.1 status OK
sensor_046 2026-03-01 00:00:00 corrente 2.3 status OK
```

Etapa 2: Versão Sequencial:

Estruturas de dados para as linhas:

```c
typedef struct {
    int id;
    double soma;
    double soma_quadrada;
    int qnt;
} Sensores;
```

```c
// Uso de calloc para inicializar array sem lixo de memória (usando malloc calculava errado o desvio padrão)
Sensores* s = (Sensores*)calloc(MAX_SENSORES, sizeof(Sensores));
```

```c
Sensores* array = (Sensores*)sensores; // Array de sensores com acesso por id
```

Foi utilizado uma struct para armazenar os dados dos sensores e combinado à um array que utiliza de acesso por ID do sensor para armazenar esses dados da struct, se assemelhando muito à estrutura de dados map/Dictionary. Considerei inserir cada dado de cada linha em uma struct em uma posição diferente de um array com tamanho igual ao número de linhas, mas além de tornar muito mais lento (para calcular cada sensor, teria que passar por cada posição do vetor verificando o ID do sensor e, caso batesse o valor do ID procurado, adicionaria em uma outra variável fora da struct), também estaria alocando muita memória para isso.

Cálculo do consumo de memória total estimado:

Para guardar o arquivo inteiro em memória exigiria aproximadamente **1,0 GB** (17.280.000 linhas × ~62 bytes por linha). Contudo isso tornaria o programa inviável em máquinas com memória limitada e desnecessário, pois as métricas que foram pedidas não exigem acesso simultâneo a todas as leituras.

A implementação utiliza um array de 1.000 structs Sensores, onde cada struct ocupa 32 bytes (incluindo padding de alinhamento). O consumo total é:

1.000 × 32 bytes = 32.000 bytes ≈ 31,3 KB

O buffer char linha[90] é reutilizado a cada iteração do loop, não acumulando memória. Isso representa uma redução de aproximadamente **33.000 vezes** em relação à abordagem de guardar todas as linhas, viabilizando o processamento de arquivos de qualquer tamanho com memória constante.

Estrutura de dados para as estatísticas:

```
// Struct para calculo de tempo e informações de sensores que deverão ser exibidas
struct timespec inicio, fim;
```

Para armazenar menos espaço, parte das informações armazenadas para o cálculo estão dentro da própria struct do sensor (principalmente porque não era necessário usar todos os dados para coletar todas as informações necessárias), entretanto, também há a struct de tempo para calcular o tempo total que demorou. Dentro da struct do sensor, para cálculo da quantidade daquele sensor, para a soma e a soma quadrado para cálculo do desvio padrão já são salvos. Para informações mais gerais, independentes de um sensor específico, foi salvo em uma variável global.

Implementação:

Para extrair os campos, é coletado a linha por linha em um loop em que pega cada linha do arquivo e insere em um array de char (tamanho de 90 posições) usando **fgets** para ser analisado. Em seguida, usando **sscanf**, é feito um exemplo de linha indicando de quais pontos devem ser retirados determinados dados como o ID, o tipo, o valor coletado pelo sensor e o status

e inserido em sua definida variável. Também é usado o **strcmp** para comparar o tipo de sensor para cálculo do consumo total de energia.

Pseudocódigo:

```
INÍCIO

abrir arquivo "sensores.log"

cria array de sensores tamanho MAX_SENSORES iniciado por 0

PARA CADA LINHA, FAÇA:
     ler id, tipo, valor, status

     sensor = sensor[id]
     sensor.soma += valor
     sensor.soma_quadrado += valor * valor
     sensor.quantidade += 1

     SE status é CRÍTICO OU ALERTA:
          total_alertas += 1
     FIMSE
FIMPARA

maior_desvio = 0
sensor_instavel = nulo

PARA CADA SENSOR DO VETOR, FAÇA:
     SE sensor.quantidade > 0:
          média = sensor.soma / sensor.quantidade
          desvio = raiz((soma_quadrada/quantidade) - (média²))
          SE desvio > maior_desvio:
               maior_desvio = desvio
               sensor_instavel = sensor
          FIMSE
     FIMSE
FIMPARA

FIM
```

O trecho mais desafiador foi, com certeza, extrair os valores necessários para responder as informações solicitadas já que demorei um pouco para compreender como coletar cada dado da linha (sabia do fgets e

sscanf mas tive que estudar como cada um funciona para escrever) e comumente coletava informações demais de cada linha, por isso, demorei para pensar em uma forma de ser conciso no consumo de memória e só pegar o necessário.

Resultados:

Tempo de execução (T_seq): 8,09 segundos
Sensor mais instável encontrado: sensor_062 (Desvio Padrão: 1499,47)
Total de alertas (ALERTA + CRÍTICO): 3.444.377
Consumo total de energia: 1.707.346.382,18 Wh

PRINT DA EXECUÇÃO:

```
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ ./sensor_analyzer_seq sensores.log

--- Média de Temperatura por Sensor (Primeiros 10) ---
Sensor_001: Média = 2.75
Sensor_002: Média = 26.51
Sensor_003: Média = 26.51
Sensor_004: Média = 26.51
Sensor_005: Média = 26.51
Sensor_006: Média = 2.75
Sensor_007: Média = 50.03
Sensor_008: Média = 493.90
Sensor_009: Média = 26.51
Sensor_010: Média = 26.51

Sensor mais instável: sensor_062 (Desvio Padrão: 1499.47)
Total de alertas: 3444377
Consumo total de energia: 1707346382.18Wh
Tempo: 8.094816 segundos
```

Etapa 3: Versão Paralela com Mutex:

Estruturas de dados para as linhas:

Foi decidido que iríamos manter a base da nossa versão sequencial para fazer a paralela, mas agora adaptado para que as threads consigam acessar os mesmos dados, controlando em que momento cada thread acessa essas informações. O vetor principal continua sendo o array de sensores que foi indexado diretamente pelo ID do sensor, o que permite localizar rapidamente a posição a ser atualizada a cada leitura do arquivo. Além dele, permanecem as variáveis globais **consumo_total**, usada para acumular o valor total de consumo registrado, e **qnt_alertas**, que contabilizam quantas leituras entraram em condição de alerta.

Nessa versão paralela, essas três estruturas deixam de ser exclusivas de um único fluxo de execução e passam a ser compartilhadas por todas as threads (via transformar elas em globais). Por isso, foi necessário proteger seu acesso com mutex, evitando que duas ou mais threads atualizem simultaneamente o mesmo

sensor ou os acumuladores globais, o que causaria uma certa inconsistência nos resultados.

Também tivemos que criar a estrutura **ThreadArgs**, usada para entregar para cada thread exatamente o bloco do arquivo que ela tem que processar. Essa estrutura contém:

- o identificador da thread, (pensamos inserir o id pra indicar a thread mas o print final deveria ser idêntico ao sequencial então no final só não usamos esse id de thread)
- o nome do arquivo a ser aberto,
- a linha inicial,
- e a linha final do intervalo atribuído.

Com isso, cada thread sabe exatamente onde deve começar e onde deve parar, sem depender de controle central durante a execução.

As variáveis que realmente exigiram proteção foram:

- o vetor **Sensores* s**, porque diferentes threads podem atualizar sensores ao mesmo tempo;
- **consumo_total**, porque cada linha válida contribui para a soma global;
- **qnt_alertas**, porque múltiplas threads podem identificar alertas simultaneamente.

```
// Variáveis globais
double consumo_total = 0;
double qnt_alertas = 0;
Sensores* s; // Agora Sensores* s é global para todas as threads terem acesso
pthread_mutex_t mutex;
```

Já as variáveis temporárias usadas durante o processamento de cada linha, tipo **id, tipo, valor, status e linha_atual,** permaneceram locais dentro da função executada por cada thread. Como cada thread possui sua própria cópia dessas variáveis, não existe compartilhamento e, portanto, não há necessidade de proteção que nem as acimas.

Divisão do trabalho:

As linhas do arquivo são distribuídas em blocos contíguos entre as threads. O programa conta o total de linhas do arquivo, ele divide pelo número de threads e atribui a cada uma um intervalo (início, fim). Cada thread então abre o arquivo de forma independente e avança até a linha inicial do seu bloco antes de começar a processar.

Exemplo:

Tamanho de cada bloco para 4 threads: 17.280.000 ÷ 4 = 4.320.000 linhas por thread.

Tamanho de cada bloco para 8 threads: 17.280.000 ÷ 8 = 2.160.000 linhas por thread.

O tratamento de sobras é feito atribuindo à última thread o restante das linhas (args[n-1].fim = total_linhas), garantindo que nenhuma linha seja perdida mesmo quando o total não é divisível pelo número de threads.

Sincronização:

O mutex é travado antes de atualizar qualquer dado compartilhado (ou seja: struct do sensor, consumo_total, qnt_alertas) e é liberado logo após. A seção crítica protegida é o bloco inteiro de atualização por linha lida. Então não há risco de deadlock porque existe apenas um mutex no programa, e ele é sempre liberado na mesma função que foi travado, assim nós garantimos que ele não ficaria travado por causa de um oversight no código.

```
// Dando lock no mutex para evitar o acesso de outra thread
pthread_mutex_lock(&mutex);

Sensores* sensor = &s[id]; // Gera sensor com as informações salvas na struct

sensor->id = id;
sensor->soma += valor;
sensor->soma_quadrada += valor * valor;
sensor->qnt += 1;

// Pega a primeira letra da string e compara, A para ALERTA e C para CRITICO
if (status[0] == 'A' || status[0] == 'C') {
    (qnt_alertas) += 1;
}

// Usando string compare para ver se o tipo é energia para somar ao consumo total
if (strcmp(tipo, "energia") == 0) {
    consumo_total += valor;
}

// Liberando mutex
pthread_mutex_unlock(&mutex);
```

Tabela com todos os tempos e speedups para cada quantidade de Threads:

Threads	Tempo (s)	Speedup S(p)	Eficiência E(p)
1	7,93	1,02	1,02
2	6,28	1,29	0,64
4	10,85	0,75	0,19
8	11,94	0,68	0,08

16	15,19	0,53	0,03

Prints dos códigos executados:

```
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ date
Tue Mar 31 12:10:18 -03 2026
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ ./sensor_analyzer_par 1 sensores.log
./sensor_analyzer_par 2 sensores.log
./sensor_analyzer_par 4 sensores.log
./sensor_analyzer_par 8 sensores.log
./sensor_analyzer_par 16 sensores.log

--- Média de Temperatura por Sensor (Primeiros 10) ---
Sensor_001: Média = 2.75
Sensor_002: Média = 26.51
Sensor_003: Média = 26.51
Sensor_004: Média = 26.51
Sensor_005: Média = 26.51
Sensor_006: Média = 2.75
Sensor_007: Média = 50.03
Sensor_008: Média = 493.90
Sensor_009: Média = 26.51
Sensor_010: Média = 26.51

Sensor mais instável: sensor_062 (Desvio Padrão: 1499.47)
Total de alertas: 3444377
Consumo total de energia: 1707346382.18Wh
Tempo: 8.342698 segundos
```

```
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ date
Tue Mar 31 12:12:35 -03 2026
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ ./sensor_analyzer_par 2 sensores.log

--- Média de Temperatura por Sensor (Primeiros 10) ---
Sensor_001: Média = 2.75
Sensor_002: Média = 26.51
Sensor_003: Média = 26.51
Sensor_004: Média = 26.51
Sensor_005: Média = 26.51
Sensor_006: Média = 2.75
Sensor_007: Média = 50.03
Sensor_008: Média = 493.90
Sensor_009: Média = 26.51
Sensor_010: Média = 26.51

Sensor mais instável: sensor_062 (Desvio Padrão: 1499.47)
Total de alertas: 3444377
Consumo total de energia: 1707346382.18Wh
Tempo: 6.276091 segundos
```

```
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ ./sensor_analyzer_par 4 sensores.log

--- Média de Temperatura por Sensor (Primeiros 10) ---
Sensor_001: Média = 2.75
Sensor_002: Média = 26.51
Sensor_003: Média = 26.51
Sensor_004: Média = 26.51
Sensor_005: Média = 26.51
Sensor_006: Média = 2.75
Sensor_007: Média = 50.03
Sensor_008: Média = 493.90
Sensor_009: Média = 26.51
Sensor_010: Média = 26.51

Sensor mais instável: sensor_062 (Desvio Padrão: 1499.47)
Total de alertas: 3444377
Consumo total de energia: 1707346382.18Wh
Tempo: 10.846598 segundos
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ date
Tue Mar 31 12:13:22 -03 2026
```

```
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ date
Tue Mar 31 12:13:22 -03 2026
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ ./sensor_analyzer_par 8 sensores.log

--- Média de Temperatura por Sensor (Primeiros 10) ---
Sensor_001: Média = 2.75
Sensor_002: Média = 26.51
Sensor_003: Média = 26.51
Sensor_004: Média = 26.51
Sensor_005: Média = 26.51
Sensor_006: Média = 2.75
Sensor_007: Média = 50.03
Sensor_008: Média = 493.90
Sensor_009: Média = 26.51
Sensor_010: Média = 26.51

Sensor mais instável: sensor_062 (Desvio Padrão: 1499.47)
Total de alertas: 3444377
Consumo total de energia: 1707346382.18Wh
Tempo: 11.936600 segundos
```

```
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ ./sensor_analyzer_par 16 sensores.log

--- Média de Temperatura por Sensor (Primeiros 10) ---
Sensor_001: Média = 2.75
Sensor_002: Média = 26.51
Sensor_003: Média = 26.51
Sensor_004: Média = 26.51
Sensor_005: Média = 26.51
Sensor_006: Média = 2.75
Sensor_007: Média = 50.03
Sensor_008: Média = 493.90
Sensor_009: Média = 26.51
Sensor_010: Média = 26.51

Sensor mais instável: sensor_062 (Desvio Padrão: 1499.47)
Total de alertas: 3444377
Consumo total de energia: 1707346382.18Wh
Tempo: 15.194388 segundos
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ date
Tue Mar 31 12:14:45 -03 2026
```

Etapa 4: Desafio - Versão Otimizada:

Estratégia de otimização:

A estratégia que nós adotamos foi o uso de dados locais por cada thread. Na versão com mutex, o maior problema é que todas as threads disputam o mesmo lock a cada linha processada, o que causa contenção: enquanto uma thread atualiza os dados globais, as demais ficam bloqueadas esperando. Isso acaba reduzindo o paralelismo efetivo, especialmente com muitas threads competindo pelo mesmo recurso.

Então a nossa solução foi simplesmente eliminar completamente o mutex durante o processamento. Cada thread recebe sua própria cópia do array de sensores (local_s), o seu próprio acumulador de consumo (local_consumo) e seu próprio contador de alertas (local_alertas), todos alocados com calloc e zerados antes do início do processamento para não dar problema de vestígios quando for ser executado. Durante todo o loop de leitura, cada thread escreve exclusivamente nos seus dados locais, sem

nenhuma disputa com outras threads, o que resolve o problema que estávamos tendo antes.

Após o pthread_join de todas as threads, o programa principal realiza a fusão (merge) dos resultados parciais no array global. Como nesse momento apenas a thread main está em execução, não há risco de race condition e nenhum mutex é necessário. O custo é um pequeno aumento no consumo de memória proporcional ao número de threads (cada uma precisa de ~23,4 KB).

Essa abordagem é superior à versão com mutex global porque remove completamente a contenção durante a fase de processamento, que é a etapa mais longa e mais paralelizável do programa. A fase de merge é sequencial mas muito rápida, pois percorre apenas 1.000 posições por thread.

Trade-offs considerados:

Complexidade vs. ganho: a versão otimizada exige modificar a struct ThreadArgs para carregar ponteiros para os arrays locais, alocar e liberar memória em cada thread, e implementar o loop de merge na main. O código ficou mais extenso e menos imediato que o mutex, mas o ganho em escalabilidade justifica-se especialmente com 16 threads onde o speedup passou de 0,53 para 1,87, o que é uma quantidade significativa de speedup e que faz, em minha opinião, valer a complexidade necessária.

Resultado inesperado com poucos threads: com 1 e 2 threads a versão otimizada foi mais lenta que a mutex (11,50s e 15,03s vs. 7,93s e 6,28s). Isso ocorre porque com poucos threads a contenção do mutex é baixa e o overhead de alocar, zerar e depois mesclar o array local supera o benefício. A versão otimizada só começa a se destacar a partir de 8 threads, quando a contenção do mutex começa a dominar, então é provado que a versão otimizada é melhor quando utilizada a partir de 8 threads.

Implementação - diferenças em relação à versão mutex:

A struct ThreadArgs foi estendida com três novos campos para suportar o padrão de acumuladores locais:

```
typedef struct {
    int thread_id;
    char* file_name;
    long inicio;
    long fim;
    // Acumuladores locais: cada thread tem sua própria cópia, sem compartilhamento
    Sensores* local_s;
    double local_consumo;
    double local_alertas;
} ThreadArgs;
```

Na main, antes de criar as threads, cada posição do vetor args recebe seu próprio array alocado com calloc (já zerado). Dentro da função da thread, todas as escritas são feitas nos arrays locais, sem nenhum lock:

```
        if (id < MAX_SENSORES) {
            // SEM MUTEX: cada thread escreve apenas no seu próprio local_s
            Sensores* sensor = &args->local_s[id]; // Gera sensor com as informações salvas na struct

            sensor->id = id;
            sensor->soma += valor;
            sensor->soma_quadrada += valor * valor;
            sensor->qnt += 1;

            // Pega a primeira letra da string e compara, A para ALERTA e C para CRITICO
            if (status[0] == 'A' || status[0] == 'C') {
                args->local_alertas += 1;
            }

            // Usando string compare para ver se o tipo é energia para somar ao consumo local
            if (strcmp(tipo, "energia") == 0) {
                args->local_consumo += valor;
            }
        }
        linha_atual++;
    }
```

Após o pthread_join de todas as threads, a main faz o merge sequencial dos resultados parciais:

```
    // Dando join nos valores calculados em extrair_valores
    for (int i = 0; i < n_threads; i++) {
        pthread_join(threads[i], NULL);
    }

    // MERGE: fusão dos resultados locais de cada thread no resultado global
    // Feito sequencialmente no main, sem necessidade de mutex (threads já terminaram)
    for (int i = 0; i < n_threads; i++) {
        consumo_total += args[i].local_consumo;
        qnt_alertas   += args[i].local_alertas;

        for (int j = 0; j < MAX_SENSORES; j++) {
            if (args[i].local_s[j].qnt > 0) {
                s[j].id = j;
                s[j].soma          += args[i].local_s[j].soma;
                s[j].soma_quadrada += args[i].local_s[j].soma_quadrada;
                s[j].qnt           += args[i].local_s[j].qnt;
            }
        }

        // Libera o array local da thread após o merge
        free(args[i].local_s);
    }

    fclose(file);
```

Tabela com todos os tempos e speedups para cada quantidade de Threads:

Threads	Tempo (s)	Speedup S(p)	Eficiência E(p)
1	11,50	0,70	0,70
2	15,03	0,54	0,27
4	8,27	0,98	0,24
8	7,59	1,07	0,13
16	4,33	1,87	0,12

É possível ver uma diferença significativa entre o paralelizado pelo mutex e o paralelizado otimizado.

Prints de Execução:

```
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ date
Tue Mar 31 12:14:45 -03 2026
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ ./sensor_analyzer_par_otimizado 1 sensores.log

--- Média de Temperatura por Sensor (Primeiros 10) ---
Sensor_001: Média = 2.75
Sensor_002: Média = 26.51
Sensor_003: Média = 26.51
Sensor_004: Média = 26.51
Sensor_005: Média = 26.51
Sensor_006: Média = 2.75
Sensor_007: Média = 50.03
Sensor_008: Média = 493.90
Sensor_009: Média = 26.51
Sensor_010: Média = 26.51

Sensor mais instável: sensor_062 (Desvio Padrão: 1499.47)
Total de alertas: 3444377
Consumo total de energia: 1707346382.18Wh
Tempo: 11.500275 segundos
```

```
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ ./sensor_analyzer_par_otimizado 2  sensores.log

--- Média de Temperatura por Sensor (Primeiros 10) ---
Sensor_001: Média = 2.75
Sensor_002: Média = 26.51
Sensor_003: Média = 26.51
Sensor_004: Média = 26.51
Sensor_005: Média = 26.51
Sensor_006: Média = 2.75
Sensor_007: Média = 50.03
Sensor_008: Média = 493.90
Sensor_009: Média = 26.51
Sensor_010: Média = 26.51

Sensor mais instável: sensor_062 (Desvio Padrão: 1499.47)
Total de alertas: 3444377
Consumo total de energia: 1707346382.18Wh
Tempo: 15.028964 segundos
```

```
Tempo: 15.028964 segundos
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ ./sensor_analyzer_par_otimizado 4  sensores.log

--- Média de Temperatura por Sensor (Primeiros 10) ---
Sensor_001: Média = 2.75
Sensor_002: Média = 26.51
Sensor_003: Média = 26.51
Sensor_004: Média = 26.51
Sensor_005: Média = 26.51
Sensor_006: Média = 2.75
Sensor_007: Média = 50.03
Sensor_008: Média = 493.90
Sensor_009: Média = 26.51
Sensor_010: Média = 26.51

Sensor mais instável: sensor_062 (Desvio Padrão: 1499.47)
Total de alertas: 3444377
Consumo total de energia: 1707346382.18Wh
Tempo: 8.266459 segundos
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ date
Tue Mar 31 12:23:05 -03 2026
```

```
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ date
Tue Mar 31 12:23:05 -03 2026
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ ./sensor_analyzer_par_otimizado 8  sensores.log

--- Média de Temperatura por Sensor (Primeiros 10) ---
Sensor_001: Média = 2.75
Sensor_002: Média = 26.51
Sensor_003: Média = 26.51
Sensor_004: Média = 26.51
Sensor_005: Média = 26.51
Sensor_006: Média = 2.75
Sensor_007: Média = 50.03
Sensor_008: Média = 493.90
Sensor_009: Média = 26.51
Sensor_010: Média = 26.51

Sensor mais instável: sensor_062 (Desvio Padrão: 1499.47)
Total de alertas: 3444377
Consumo total de energia: 1707346382.18Wh
Tempo: 7.586652 segundos
```

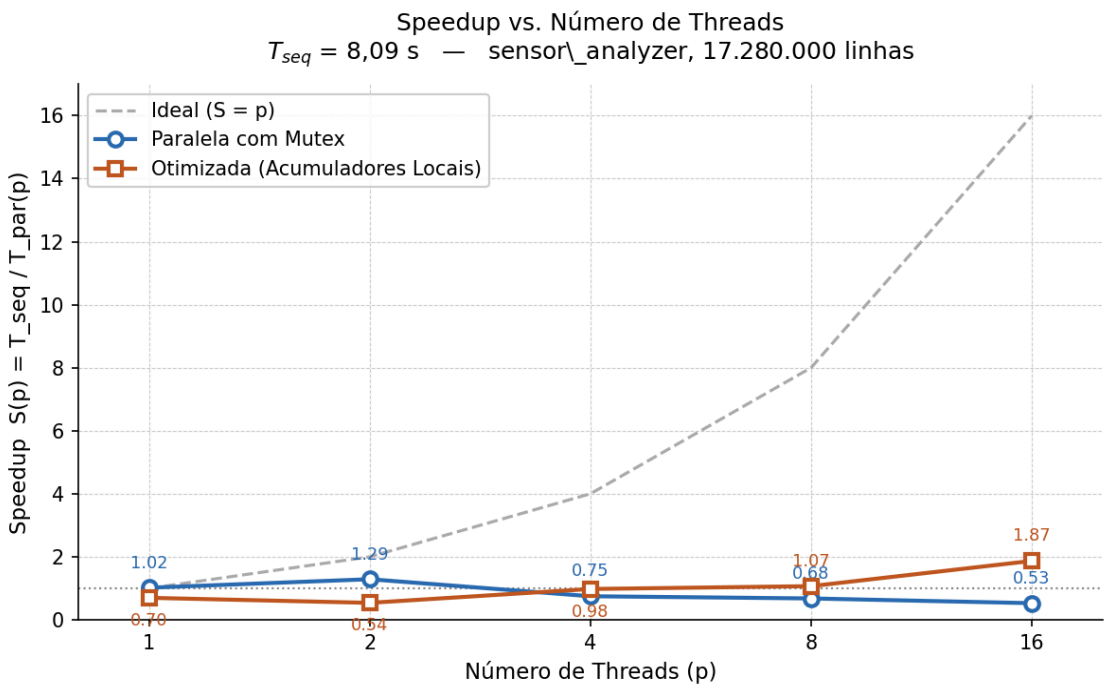
```
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ ./sensor_analyzer_par_otimizado 16 sensores.log

--- Média de Temperatura por Sensor (Primeiros 10) ---
Sensor_001: Média = 2.75
Sensor_002: Média = 26.51
Sensor_003: Média = 26.51
Sensor_004: Média = 26.51
Sensor_005: Média = 26.51
Sensor_006: Média = 2.75
Sensor_007: Média = 50.03
Sensor_008: Média = 493.90
Sensor_009: Média = 26.51
Sensor_010: Média = 26.51

Sensor mais instável: sensor_062 (Desvio Padrão: 1499.47)
Total de alertas: 3444377
Consumo total de energia: 1707346382.18Wh
Tempo: 4.329896 segundos
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ date
Tue Mar 31 12:49:08 -03 2026
```

Análise Comparativa e Discussão:

Gráfico Comparativo:

Ao observar o gráfico, é notável que a versão com mutex apresenta baixa escalabilidade conforme o número de threads cresce, sendo esse comportamento explicado pela Lei de Amdahl, pela utilização do mutex aumentar a fração sequencial do código e, assim, limitar o ganho que o paralelismo poderia trazer.

Já a versão otimizada, mesmo que haja a queda de eficiência com o aumento do número de threads, o desempenho geral se mantém superior, indicando que esta versão possui melhor escalabilidade, permitindo que fração maior do código possa ser paralelizada.



Lei de Amdahl e fração paralelizável do código:

A Lei de Amdahl define o speedup máximo teórico como S(p) = 1 / ((1−P) + P/p), onde P é a fração paralelizável do código. Aplicando aos resultados obtidos:

Versão mutex (melhor ponto: S=1,29 com 2 threads):

```
S(p) = 1 / ((1-P) + P/p)
1,29 = 1 / ((1-P) + P/2)
1,29 × (1 - P + P/2) = 1
1,29 - 0,645P = 1   →   P ≈ 0,45
```

Versão otimizada (melhor ponto: S=1,87 com 16 threads):

```
S(p) = 1 / ((1-P) + P/p)
1,87 = 1 / ((1-P) + P/16)
1,87 × (1 - P + P/16) = 1
1,87 - 1,753P = 1   →   P ≈ 0,50
```

Com esses resultados, é possível notar que, para a versão do mutex, apenas 45% do código é eficientemente paralelizado (como dito antes, isso se deve ao fato da utilização do mutex), limitando o ganho de desempenho.

Já na versão otimizada, apesar de maior, apenas 50% do código foi paralelizada de fato. Os resultados indicam que a versão otimizada ainda possui margem para melhorias caso seja possível aumentar a fração paralelizável do código, porém, também fica evidente que existe um limite teórico para o ganho de desempenho dependente da fração sequencial que, caso seja reduzida, também aumentaria o speedup

Análise de escalabilidade:

A eficiência foi calculada para cada configuração utilizando E(p) = S(p)/p, conforme apresentado nas tabelas das Etapas 3 e 4.

Na **versão com mutex**, a eficiência cai abaixo de 0,7 já a partir de **2 threads** (E = 0,64). Isso ocorre porque o overhead de contenção do mutex, tempo que cada thread passa esperando o lock ficar disponível, cresce proporcionalmente ao número de threads, reduzindo o aproveitamento real do paralelismo desde as configurações mais simples.

Na **versão otimizada**, a eficiência também cai abaixo de 0,7 a partir de **2 threads** (E = 0,27 com 2 threads; com 1 thread atinge exatamente o limite de 0,70). Mesmo sem mutex, a queda pode ser explicada pela fração sequencial do código que permanece constante: a contagem inicial de linhas, o avanço até o bloco de cada thread e o merge final que não são paralelizados. Pela Lei de Amdahl, com 50% de código paralelizável, o speedup máximo teórico é 2.0, o que limita a eficiência abaixo de 0,7 com qualquer número de threads acima de 1.

Isso indica que ambas as versões apresentam **escalabilidade limitada** para esse problema específico, com a versão otimizada sendo superior em desempenho absoluto mas igualmente afetada pela parcela sequencial inevitável do código.

Comparação entre as versões:

Com base nos testes realizados, observa-se que a versão com mutex apresentou melhora ao utilizar 1 thread (7,93s) e 2 threads (6,28s). No entanto, ao aumentar o número de threads (4, 8 e 16), houve degradação no desempenho, chegando a 15,19s com 16 threads. Esse comportamento está relacionado ao overhead gerado pelo uso do mutex, já que múltiplas threads passam a disputar o mesmo recurso, reduzindo a eficiência da paralelização.

Comparando com a versão sequencial (8,09s), nota-se que, nesse cenário, a versão paralela com mutex se torna menos eficiente, evidenciando que o aumento do número de threads nem sempre resulta em melhor desempenho.

Já na versão otimizada, a utilização de variáveis locais eliminou a necessidade de uso de mutex durante o processamento. Com isso, foi possível reduzir significativamente o overhead causado pela sincronização, permitindo melhor aproveitamento das threads e resultando em maior eficiência, especialmente em cenários com maior nível de paralelismo.

Ao observar o gráfico, é notável que a versão com mutex apresenta baixa escalabilidade conforme o número de threads cresce, sendo esse comportamento explicado pela Lei de Amdahl, pela utilização do mutex aumentar a fração sequencial do código e, assim, limitar o ganho que o paralelismo poderia trazer.

Já a versão otimizada, mesmo que haja a queda de eficiência com o aumento do número de threads, o desempenho geral se mantém superior, indicando que esta

versão possui melhor escalabilidade, permitindo que fração maior do código possa ser paralelizada.

Gargalos identificados:

Dois gargalos principais foram identificados. O primeiro é a contenção de mutex: na versão com mutex, todas as threads disputam o mesmo lock a cada linha, serializando o acesso aos dados compartilhados. O segundo é o gargalo de I/O: cada thread abre o arquivo independentemente e percorre linha por linha até chegar ao bloco, gerando leitura redundante de disco. Uma melhoria futura seria usar offsets em bytes com fseek para posicionar cada thread diretamente no início do seu bloco, eliminando a leitura sequencial desnecessária.

Conclusão:

Com este projeto, foi possível compreender visualmente pelas comparações o ganho de desempenho que a paralelização traz ao rodar um código, mas que também, deve-se pensar bastante em como realizar esse procedimento de paralelização para que não resulte em uma eficiência até pior que versões sequenciais do mesmo script.

As maiores dificuldades desse projeto vieram na construção de scripts eficientes, inclusive durante o desenvolvimento da versão paralela, por exemplo ao inserir o mutex dentro do loop, a eficiência caiu drasticamente e, por falta de verificadores como mensagens via terminal para debug, demoramos para compreender e corrigir o erro, só se resolvendo depois de muitos testes quando compreendemos o melhor local para inserir o mutex na função, fomos ver só depois que ele já estava inicializando no começo do loop o que piorou a eficiência do código, então ajustamos para que ele só abre quando é para salvar os dados, melhorando assim sua eficiência.

Uma possível melhoria na situação de implementar novamente esse código, seria a inserção de mensagens via terminal para facilitar o debug e realização constante de testes para otimizar meu tempo e facilitar o encontro de erros/bugs.

Com isso, esse trabalho nos mostrou que é possível dividir processos maiores em processos menores e distribuir melhor o trabalho aproveitando da arquitetura multicore dos processadores, expandindo minha visão sobre como utilizar da ferramenta de paralelização por thread.

Apêndice: Prints de Execução Completos

Todos os prints contêm a data/hora do sistema visível (comando date executado antes de cada teste).

Geração do arquivo:

```
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ python3 scriptProfessor/generate_sensor_log.py --sensores 100 --dias 2 --output sensores.log
============================================================
GERADOR DE LOGS DE SENSORES IoT PARA DATA CENTER
============================================================
Gerando log com 100 sensores por 2 dia(s)...
Total estimado de linhas: 17,280,000

Distribuição dos sensores:
  corrente: 14 sensores
  temperatura: 41 sensores
  umidade: 24 sensores
  energia: 20 sensores
  pressao: 1 sensores

Gerando arquivo: sensores.log

Dia 1/2...
  Hora 24/24: 100%

Dia 2/2...
  Hora 24/24: 100%


Arquivo gerado com sucesso!
Total de linhas: 17,280,000
Tamanho do arquivo: 934.76 MB
Tempo de geração: 98.32 segundos
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ date
Tue Mar 31 12:06:27 -03 2026
```

Primeiras 10 linhas do arquivo (head -10):

```
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ date
Tue Mar 31 12:06:27 -03 2026
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ head -10 sensores.log
sensor_001 2026-03-01 00:00:00 corrente 2.9 status OK
sensor_006 2026-03-01 00:00:00 corrente 2.0 status OK
sensor_012 2026-03-01 00:00:00 corrente 2.3 status OK
sensor_015 2026-03-01 00:00:00 corrente 2.5 status OK
sensor_030 2026-03-01 00:00:00 corrente 2.8 status OK
sensor_033 2026-03-01 00:00:00 corrente 3.0 status OK
sensor_040 2026-03-01 00:00:00 corrente 3.9 status OK
sensor_042 2026-03-01 00:00:00 corrente 3.5 status OK
sensor_045 2026-03-01 00:00:00 corrente 3.1 status OK
sensor_046 2026-03-01 00:00:00 corrente 2.3 status OK
```

Versão Sequencial:

```
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ ./sensor_analyzer_seq sensores.log

--- Média de Temperatura por Sensor (Primeiros 10) ---
Sensor_001: Média = 2.75
Sensor_002: Média = 26.51
Sensor_003: Média = 26.51
Sensor_004: Média = 26.51
Sensor_005: Média = 26.51
Sensor_006: Média = 2.75
Sensor_007: Média = 50.03
Sensor_008: Média = 493.90
Sensor_009: Média = 26.51
Sensor_010: Média = 26.51

Sensor mais instável: sensor_062 (Desvio Padrão: 1499.47)
Total de alertas: 3444377
Consumo total de energia: 1707346382.18Wh
Tempo: 8.094816 segundos
```

Versão Paralela com Mutex 1 thread:

```
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ date
Tue Mar 31 12:10:18 -03 2026
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ ./sensor_analyzer_par 1 sensores.log
./sensor_analyzer_par 2 sensores.log
./sensor_analyzer_par 4 sensores.log
./sensor_analyzer_par 8 sensores.log
./sensor_analyzer_par 16 sensores.log

--- Média de Temperatura por Sensor (Primeiros 10) ---
Sensor_001: Média = 2.75
Sensor_002: Média = 26.51
Sensor_003: Média = 26.51
Sensor_004: Média = 26.51
Sensor_005: Média = 26.51
Sensor_006: Média = 2.75
Sensor_007: Média = 50.03
Sensor_008: Média = 493.90
Sensor_009: Média = 26.51
Sensor_010: Média = 26.51

Sensor mais instável: sensor_062 (Desvio Padrão: 1499.47)
Total de alertas: 3444377
Consumo total de energia: 1707346382.18Wh
Tempo: 8.342698 segundos
```

Versão Paralela com Mutex 2 threads:

```
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ date
Tue Mar 31 12:12:35 -03 2026
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ ./sensor_analyzer_par 2 sensores.log

--- Média de Temperatura por Sensor (Primeiros 10) ---
Sensor_001: Média = 2.75
Sensor_002: Média = 26.51
Sensor_003: Média = 26.51
Sensor_004: Média = 26.51
Sensor_005: Média = 26.51
Sensor_006: Média = 2.75
Sensor_007: Média = 50.03
Sensor_008: Média = 493.90
Sensor_009: Média = 26.51
Sensor_010: Média = 26.51

Sensor mais instável: sensor_062 (Desvio Padrão: 1499.47)
Total de alertas: 3444377
Consumo total de energia: 1707346382.18Wh
Tempo: 6.276091 segundos
```

Versão Paralela com Mutex 4 threads:

```
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ ./sensor_analyzer_par 4 sensores.log

--- Média de Temperatura por Sensor (Primeiros 10) ---
Sensor_001: Média = 2.75
Sensor_002: Média = 26.51
Sensor_003: Média = 26.51
Sensor_004: Média = 26.51
Sensor_005: Média = 26.51
Sensor_006: Média = 2.75
Sensor_007: Média = 50.03
Sensor_008: Média = 493.90
Sensor_009: Média = 26.51
Sensor_010: Média = 26.51

Sensor mais instável: sensor_062 (Desvio Padrão: 1499.47)
Total de alertas: 3444377
Consumo total de energia: 1707346382.18Wh
Tempo: 10.846598 segundos
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ date
Tue Mar 31 12:13:22 -03 2026
```

Versão Paralela com Mutex 8 threads:

```
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ date
Tue Mar 31 12:13:22 -03 2026
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ ./sensor_analyzer_par 8 sensores.log

--- Média de Temperatura por Sensor (Primeiros 10) ---
Sensor_001: Média = 2.75
Sensor_002: Média = 26.51
Sensor_003: Média = 26.51
Sensor_004: Média = 26.51
Sensor_005: Média = 26.51
Sensor_006: Média = 2.75
Sensor_007: Média = 50.03
Sensor_008: Média = 493.90
Sensor_009: Média = 26.51
Sensor_010: Média = 26.51

Sensor mais instável: sensor_062 (Desvio Padrão: 1499.47)
Total de alertas: 3444377
Consumo total de energia: 1707346382.18Wh
Tempo: 11.936600 segundos
```

Versão Paralela com Mutex 16 threads:

```
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ ./sensor_analyzer_par 16 sensores.log

--- Média de Temperatura por Sensor (Primeiros 10) ---
Sensor_001: Média = 2.75
Sensor_002: Média = 26.51
Sensor_003: Média = 26.51
Sensor_004: Média = 26.51
Sensor_005: Média = 26.51
Sensor_006: Média = 2.75
Sensor_007: Média = 50.03
Sensor_008: Média = 493.90
Sensor_009: Média = 26.51
Sensor_010: Média = 26.51

Sensor mais instável: sensor_062 (Desvio Padrão: 1499.47)
Total de alertas: 3444377
Consumo total de energia: 1707346382.18Wh
Tempo: 15.194388 segundos
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ date
Tue Mar 31 12:14:45 -03 2026
```

Versão Otimizada (Dados Locais) 1 thread:

```
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ date
Tue Mar 31 12:14:45 -03 2026
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ ./sensor_analyzer_par_otimizado 1 sensores.log

--- Média de Temperatura por Sensor (Primeiros 10) ---
Sensor_001: Média = 2.75
Sensor_002: Média = 26.51
Sensor_003: Média = 26.51
Sensor_004: Média = 26.51
Sensor_005: Média = 26.51
Sensor_006: Média = 2.75
Sensor_007: Média = 50.03
Sensor_008: Média = 493.90
Sensor_009: Média = 26.51
Sensor_010: Média = 26.51

Sensor mais instável: sensor_062 (Desvio Padrão: 1499.47)
Total de alertas: 3444377
Consumo total de energia: 1707346382.18Wh
Tempo: 11.500275 segundos
```

Versão Otimizada 2 threads:

```
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ ./sensor_analyzer_par_otimizado 2  sensores.log

--- Média de Temperatura por Sensor (Primeiros 10) ---
Sensor_001: Média = 2.75
Sensor_002: Média = 26.51
Sensor_003: Média = 26.51
Sensor_004: Média = 26.51
Sensor_005: Média = 26.51
Sensor_006: Média = 2.75
Sensor_007: Média = 50.03
Sensor_008: Média = 493.90
Sensor_009: Média = 26.51
Sensor_010: Média = 26.51

Sensor mais instável: sensor_062 (Desvio Padrão: 1499.47)
Total de alertas: 3444377
Consumo total de energia: 1707346382.18Wh
Tempo: 15.028964 segundos
```

Versão Otimizada 4 threads:

```
Tempo: 15.028964 segundos
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ ./sensor_analyzer_par_otimizado 4  sensores.log

--- Média de Temperatura por Sensor (Primeiros 10) ---
Sensor_001: Média = 2.75
Sensor_002: Média = 26.51
Sensor_003: Média = 26.51
Sensor_004: Média = 26.51
Sensor_005: Média = 26.51
Sensor_006: Média = 2.75
Sensor_007: Média = 50.03
Sensor_008: Média = 493.90
Sensor_009: Média = 26.51
Sensor_010: Média = 26.51

Sensor mais instável: sensor_062 (Desvio Padrão: 1499.47)
Total de alertas: 3444377
Consumo total de energia: 1707346382.18Wh
Tempo: 8.266459 segundos
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ date
Tue Mar 31 12:23:05 -03 2026
```

Versão Otimizada 8 threads:

```
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ date
Tue Mar 31 12:23:05 -03 2026
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ ./sensor_analyzer_par_otimizado 8  sensores.log

--- Média de Temperatura por Sensor (Primeiros 10) ---
Sensor_001: Média = 2.75
Sensor_002: Média = 26.51
Sensor_003: Média = 26.51
Sensor_004: Média = 26.51
Sensor_005: Média = 26.51
Sensor_006: Média = 2.75
Sensor_007: Média = 50.03
Sensor_008: Média = 493.90
Sensor_009: Média = 26.51
Sensor_010: Média = 26.51

Sensor mais instável: sensor_062 (Desvio Padrão: 1499.47)
Total de alertas: 3444377
Consumo total de energia: 1707346382.18Wh
Tempo: 7.586652 segundos
```

Versão Otimizada 16 threads:

```
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ ./sensor_analyzer_par_otimizado 16 sensores.log

--- Média de Temperatura por Sensor (Primeiros 10) ---
Sensor_001: Média = 2.75
Sensor_002: Média = 26.51
Sensor_003: Média = 26.51
Sensor_004: Média = 26.51
Sensor_005: Média = 26.51
Sensor_006: Média = 2.75
Sensor_007: Média = 50.03
Sensor_008: Média = 493.90
Sensor_009: Média = 26.51
Sensor_010: Média = 26.51

Sensor mais instável: sensor_062 (Desvio Padrão: 1499.47)
Total de alertas: 3444377
Consumo total de energia: 1707346382.18Wh
Tempo: 4.329896 segundos
moyak@MKM:~/Projeto1-Computacao-Paralela-main$ date
Tue Mar 31 12:49:08 -03 2026
```